

End-to-End Data & Experiment Pipeline for the SIC Project (multi-hop Semantic Induction Circuits)

Working plan — based on Ren et al. 2024 and the project proposal

September 2026

This document lists everything we need — data, models, tooling, and processing steps — in the order we should build it, adapted from the experimental setup of [Ren et al. 2024 \(Identifying Semantic Induction Heads\)](#) to our research question (distributed Semantic Induction Circuits for multi-hop relations).

1 Pipeline overview (build in this order)

1. Choose model suite + checkpoints; verify TransformerLens support.
2. Generate synthetic multi-hop ICL datasets (gold relations known by construction).
3. Tokenize and anchor: map every entity to its token indices; fix a multi-token convention.
4. Run with activation caching; compute per-head Relation Index (RI), reproducing Ren et al. on 1-hop as a sanity check.
5. Generalize RI to circuits; run activation / path patching; measure faithfulness of the sparse subgraph.
6. Sweep across training checkpoints; separate format-compliance failures from pattern-discovery failures.

2 Models and checkpoints

Ren et al. used InternLM2-1.8B (24 layers $\times$ 16 heads) for the head analysis, and trained their own 1.1B model on SlimPajama with checkpoints every 200 steps for the ICL-emergence experiments. We do *not* train: we use public checkpoint suites.

- **Pythia** (EleutherAI): 154 checkpoints per model size, fully public training data order. Repo: <https://github.com/EleutherAI/pythia>; models on HF, e.g. `pythia-160m ... pythia-1b` (checkpoints via the `revision` argument, e.g. `revision="step3000"`).
- **OLMo** (AI2): <https://github.com/allenai/OLMo>; checkpoints on HF.
- **LMEnt** suite (named in our proposal): locate via [arXiv](#) / [HF search](#) and confirm checkpoint availability before committing to it.
- **Tooling**: [TransformerLens](#) — check the [supported-models list](#); Pythia and OLMo are supported. All caching and patching goes through `run_with_cache` and hook points.

Decision to make early: model size vs. number of checkpoints vs. storage. Attention patterns are $L \times H \times n^2$ per prompt *per checkpoint*; compute RI on the fly and store only per-head statistics, never raw patterns for the full sweep.

3 Datasets

3.1 What Ren et al. used (our 1-hop reference)

- **AGENDA** (Koncel-Kedziorski et al. 2019): 1,000 test samples; each = a scientific abstract + a knowledge graph of entities and typed relations, *anchored in the text*. Used for knowledge-graph relations.
- Syntactic dependencies: sentences split from the same paragraphs, parsed with **spaCy**. Note: these gold labels are themselves model output (parser noise) — a caveat they inherit and we avoid.
- ICL-levels tasks: few-shot classification prompts (their Table 1); format compliance = the forced single generated token has the right *type* (e.g. a digit), pattern discovery = the label is *correct*; class-balanced shots.

3.2 What we build (multi-hop, synthetic)

Generated from templates so that gold relations are known by construction — no parser, no annotation noise.

- **Template sources:** **CLUTRR** (compositional kinship reasoning; built for exactly this) and **bAbI** tasks. Per the proposal, replace all real nouns with fictitious entities so the model cannot answer from parametric memory.
- **Each sample must carry:** the prompt text; the list of gold triples (head, relation, tail); for multi-hop, the chain (e.g. $A \subset B, B \subset C \Rightarrow A \subset C$) with the *bridge entity* marked; character-level spans of every entity occurrence.
- **Matched difficulty tiers:** 1-hop (reproduces Ren et al.’s setting), 2-hop, optionally 3-hop — same entities, same format, so hop count is the only manipulated variable.
- **Controls to generate alongside** (needed later for patching and for RI baselines): corrupted prompts (bridge entity replaced — breaks the chain while preserving format); shuffled-relation prompts; and Ren et al.’s trick of a *fake tail* baseline (they re-computed RI with the tail forced to an arbitrary token — e.g. the 10th token — as a null distribution).
- **Anti-confound rule:** the query pair must never appear verbatim as an in-context example — otherwise a plain (non-semantic) induction head solves the task by literal copying and inflates our results.
- **Class balance:** when shots $\geq$ number of answer classes, include at least one example per class (Ren et al.’s protocol), so that format accuracy is well-defined.

4 Tokenization and anchoring

The single most error-prone step. Annotations live on words/entities; the model lives on sub-word tokens.

1. Tokenize each prompt with the *suite’s own tokenizer*; build a map from every entity occurrence (character span) to a contiguous range of token indices.
2. Fix a convention for multi-token entities (fictitious names will split into several tokens). Ren et al. operate at the token level; for spans, the common convention is to use the *last* token of the entity as its representative (where its identity is complete). Document the choice and test robustness to it.
3. Store, per sample: (prompt, token_ids, {entity → [token idx range]}, gold triples, chain structure) — e.g. one JSONL line per sample.

5 Metrics

5.1 Relation Index (Ren et al., §3) — reproduce first on 1-hop

Two stages, per head h , per gold triple with head token t_s , tail token t_o , evaluated at current tokens t_j with $j \geq \max(s, o)$:

1. **QK stage (attention selection)**: the head must attend from t_j to t_s dominantly: $s = \arg \max_k A_{j,k}^h$ and $A_{j,s}^h / \max_{k \neq s} A_{j,k}^h > \tau$ (they use $\tau = 2.2$; see their Appendix A). Otherwise the head is skipped for this triple.
2. **OV stage (tail promotion)**: project the head’s OV output to the vocabulary, $p^{h,j} = \text{softmax}(x_j W_{OV}^h W_U)$; keep only above-mean probabilities $q_{t_k}^{h,j} = \max(0, p_{t_k}^{h,j} - E_{k'}[p_{t_k'}^{h,j}])$; the score is the tail’s share $a_T^{h,j} = q_{t_o}^{h,j} / \sum_k q_{t_k}^{h,j}$. Average over j and over triples.

Our extension (core novelty): generalize RI from a single head to a *circuit*: for 2-hop, no single head needs to link A to C ; instead head h_1 may resolve $A \rightarrow B$ and write it to the residual stream, and h_2 read it and resolve $\rightarrow C$ (composition). The circuit-level RI must therefore score (i) per-edge behavior and (ii) the causal link between the heads — which is what path patching measures (below), so the metric and the intervention should be designed together.

5.2 ICL levels (for the checkpoint sweep)

Follow Ren et al. §4: **loss reduction** — interval-averaged version of Olsson et al.’s score (mean loss over tokens $j \dots i+j$ minus mean over $0 \dots i$); **format compliance** — forced single-token generation, check the token’s type; **pattern discovery** — label accuracy. Report all three per checkpoint so that “no SIC yet” is never confused with “model cannot even comply with the format yet”.

6 Causal experiments

To learn/implement (in this order):

1. **Activation patching** (node-level): replace a head’s/MLP’s activation with its value from the corrupted prompt; measure the drop in the correct-answer probability. Primer: [Heimersheim & Nanda 2024](#), “How to use and interpret activation patching”; origins: [ROME / causal tracing](#).
2. **Path patching** (edge-level): patch only along the path from component X to component Y — this is what substantiates “ h_1 feeds h_2 ”, i.e. the circuit claim. Canonical use: [Wang et al. 2022 \(IOI\)](#) — near-mandatory reading; our faithfulness protocol is modeled on it.

3. **Faithfulness** (proposal §3): run the model with only the identified sparse subgraph active (everything else mean-ablated or frozen); compare correct-answer probability to the full model. Define the recovery threshold *in advance*. Rigor ceiling for this kind of claim: [causal scrubbing](#) (skim).

7 Sanity checks and baselines

- Reproduce Ren et al. qualitatively on 1-hop relations before any 2-hop work (small scale: a few relations, a few hundred samples).
- Fake-tail / random-tail RI baseline (their trick) as the null distribution for every RI heatmap.
- Plain induction-head baseline: measure prefix-matching and copying scores (Olsson et al.) on the same heads, to show SIC heads are not just literal-copy heads — and vice versa.
- Random / earliest-checkpoint model as a floor for every metric.
- Verify the anti-confound rule held (no verbatim query repetition) by scanning generated prompts.

8 Storage and compute notes

~100–1,000 prompts $\times$ dozens of checkpoints: cache per-head *scores*, not raw attention ($L H n^2$ floats per prompt adds up fast); batch prompts per checkpoint load (loading a checkpoint dominates wall-time); models $\leq 1\text{B}$ fit the studentkillable Slurm partition per the proposal.

9 Key resources (one place)

- Ren et al. 2024 (SIH): <https://arxiv.org/abs/2402.13055>
- Olsson et al. 2022 (induction heads): <https://transformer-circuits.pub/2022/in-context-learning-a-index.html>
- Elhage et al. 2021 (framework): <https://transformer-circuits.pub/2021/framework/index.html>
- Wang et al. 2022 (IOI, path patching): <https://arxiv.org/abs/2211.00593>
- Activation patching guide: <https://arxiv.org/abs/2404.15255>
- TransformerLens: <https://github.com/TransformerLensOrg/TransformerLens>
- Pythia: <https://github.com/EleutherAI/pythia> OLMo: <https://github.com/allenai/OLMo>
- CLUTRR: <https://github.com/facebookresearch/clutrr> bAbI: <https://research.facebook.com/downloads/babi/>